

# CSC1029 OO Programming

## Lecture 14 – An Introduction to Class Associations





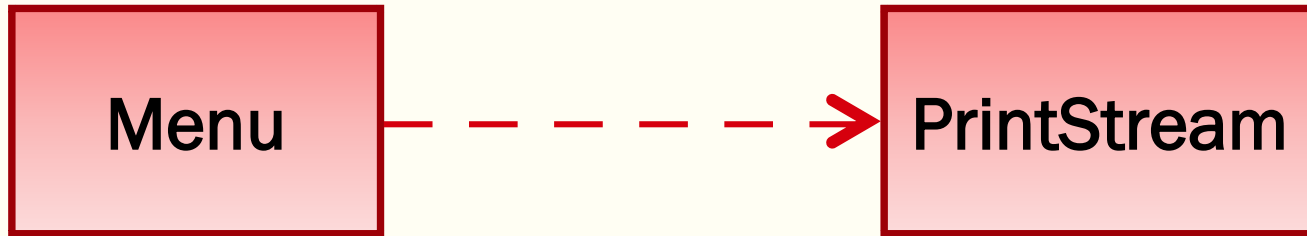
# What's to come in today's lecture?

- Class Associations
- Another look at the Student class

# Dependencies Between Classes

---

We have seen in previous lectures how one class can be *dependent* on another:



If many classes from a (problem domain) depend on one another – *coupling* is said to be *high*.

**Q:** Why is this a potential problem?

**A:** Updates to a class may require other classes that depend on it to be updated as well.

So it would be a good idea (in any design) to remove any unnecessary coupling.

# Student Class Revisited

---

## Student

- sNumber : int  
- fName : String  
- lName : String  
- modules : String  
- moduleCount : int  
- MAX\_MODULES : int

+ Student(int,String,String)  
+ getName() : String  
+ getNumber() : int  
+ getModules() : String  
+ setForename(String):void  
+ setSurname(String):void  
+ addModule(String):Boolean  
+ display():String

Does this class represent a *single* concept?

The concept of a *Module* – is buried in this definition – in a String.

In reality, there are many more pieces of data required to describe a Module.

- Module *Name*
- Module *Code*
- Module *Description*
- Module *Cats*
- Module *Assessment*
- Module *Hours*

# We really need a Module class

---

| Module                                                                                                                                                                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>- code : String</li><li>- name : String</li><li>- description : String</li><li>- catPoints : int</li><li>- hours : int</li></ul>                                                                     |
| <ul style="list-style-type: none"><li>+ Module(String,String,<br/>String,int,int)</li><li>+ getName() : String</li><li>+ getCode() : String</li><li>+ getDescription() : String</li><li>+ getCats():int</li><li>+ getHours():int</li></ul> |

In reality, there are many more pieces of data required to describe a Module.

- Module *Name*
- Module *Code*
- Module *Description*
- Module *Cats*
- Module *Hours*

What behaviour would be appropriate?

- Constructor(s)
- Accessors
- Mutators

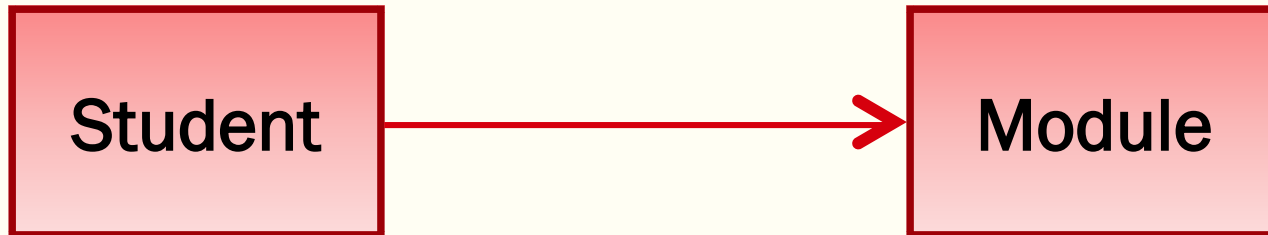
# Defining Class Relationships <sup>(1)</sup>

---

What defines the relationship between the *Student* and *Modules* classes?

In *UML*, the most basic form of relationship is called an *association*.

If two classes need to communicate, there must be a link between them. This link represents the *association*.



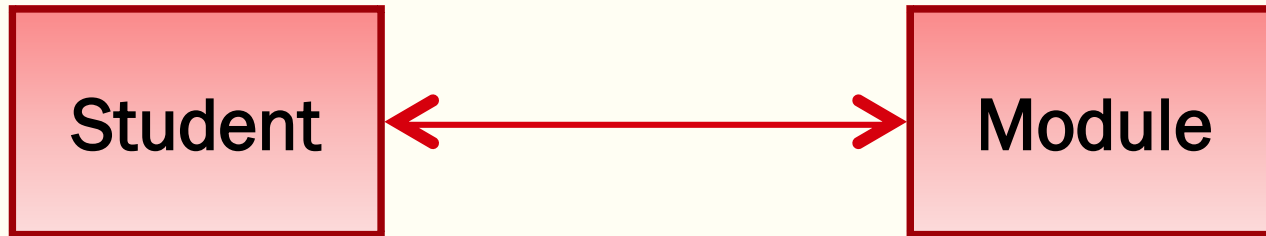
The line indicates an association between *Student* and *Module*.  
An arrow (if included) indicates the *direction* of the association.

This means that *Student* will refer to (or make use of) *Module* – but the converse is not the case.

# Defining Class Relationships (2)

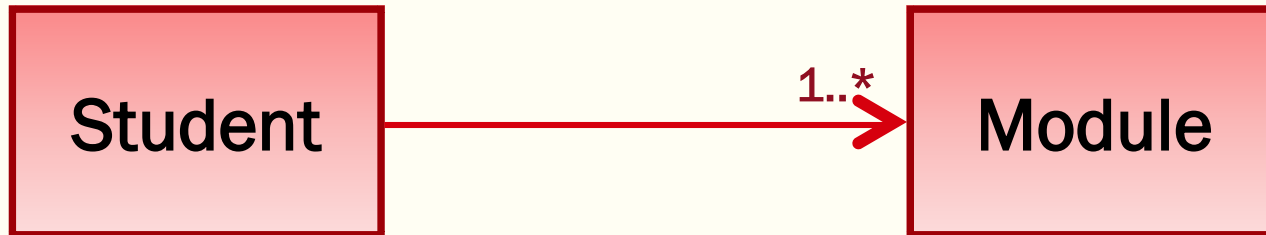
---

Arrows at both ends means a bidirectional association.



In this case, the classes will refer to one another.

The absence of arrows is also taken to mean that the association works in both directions.



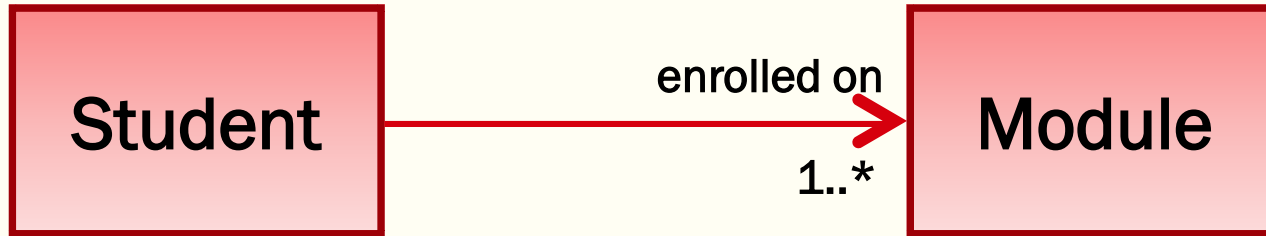
Multiplicity annotations give more information on the association ...

A single ***Student*** can associate with multiple ***Modules***.

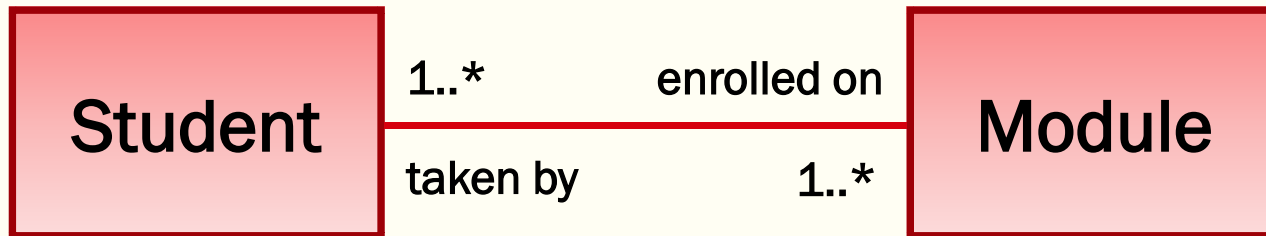
# Defining Class Relationships <sup>(3)</sup>

---

Role names may be included to indicate the behaviour of an object in an association.



A *Student* is enrolled on one or more *Modules*.



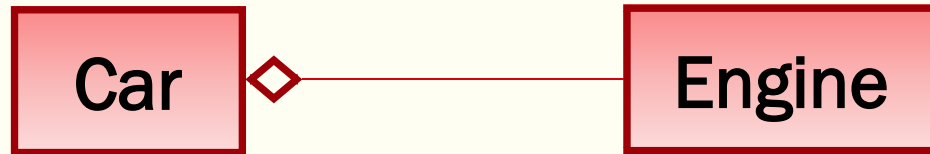
A *Module* is taken by 1 or more *Students*.



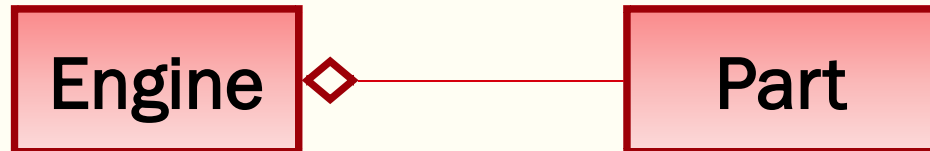
# Aggregation & Composition Associations (1)

**Aggregation** is a special form of association that specifies a whole-part relationship between the aggregate (the whole) and a component (the part).

Typically one class will represent a larger thing which consists of (or is responsible for) smaller things.



“A Car has an Engine” (1..1)

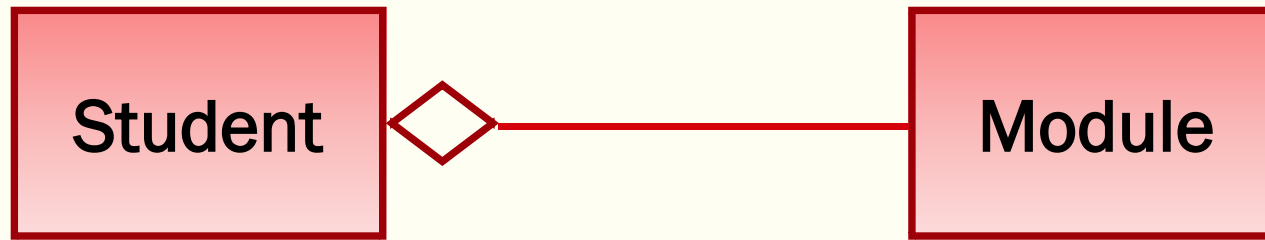


“An Engine has Parts” (1..\*)

# Aggregation & Composition Associations (2)

---

**Aggregation**:- implies that one class can logically exist independently of the other



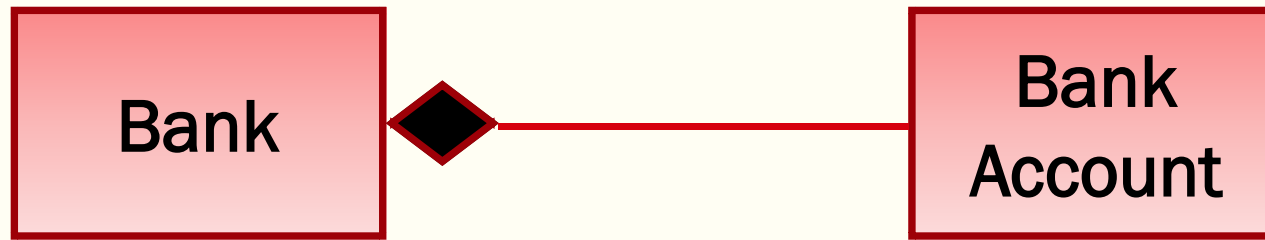
In other words, should a ***Student*** object be deleted – this ***does not mean*** that its associated ***Module*** objects will be deleted.

The ***Module*** objects will continue to exist independently.

# Aggregation & Composition Associations (3)

---

**Composition**:- is a form of aggregation with strong ownership implications - implies that objects from a class **cannot exist** independently of the other.



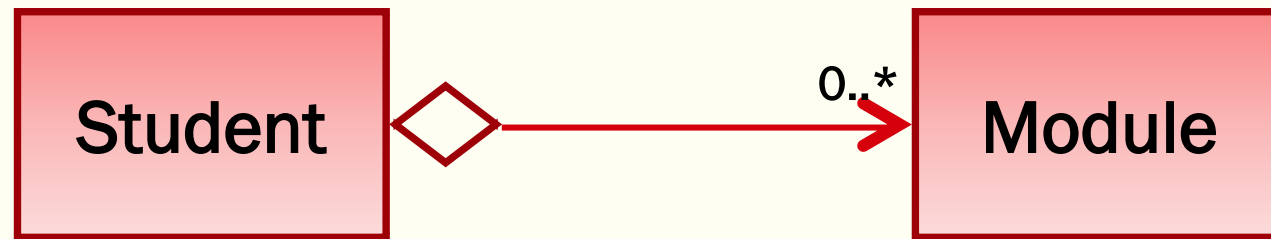
In other words, should a **Bank** object be deleted – the associated **BankAccount** objects must also be deleted.

The **BankAccount** objects do not exist independently of the associated **Bank** object.

# The Student-Module Association <sup>(1)</sup>

---

The relationship between *Student* and *Module* – is it composition or aggregation?



Must be aggregation – as they can exist independently.

From the diagram – Student class will reference the Module class (one way).

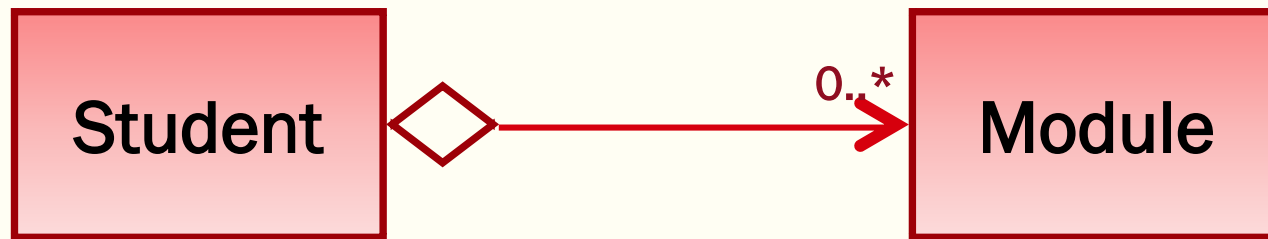
What changes will be required within the Student class?

How can such relationships be implemented?

# The Student-Module Association (2)

---

Let's implement this in Eclipse.



Must be aggregation – as they can exist independently.

For this example – what options for implementation?  
Collections – array, *ArrayList*?





# What's Next?

## More on class associations